Architecting and Containerizing GreenRoots: A Full-Stack SOA Journey with Docker, CI/CD, and Spring Boot

By Dheeraj • Student ID: 2400032903

Building modern software is no longer just about writing business logic in code. In today's cloud-native landscape, the true engineering challenge lies in **Service-Oriented Architecture (SOA)**: designing loosely coupled services, establishing seamless API contracts, packaging polyglot runtimes into isolated containers, and automating build-to-deployment pipelines through Continuous Integration and Continuous Delivery (CI/CD).

As part of my **Service-Oriented Architecture & Full-Stack Development** course, I developed and containerized **GreenRoots**—an end-to-end event management microservice dedicated to tracking community afforestation drives and environmental impact.

While the application features an intuitive React frontend and a robust Spring Boot REST backend, **the core objective and primary moto of this project was mastering containerization and DevOps automation**:

- Engineering lightweight, multi-stage **Dockerfiles**.
- Automating builds and image publishing via **GitHub Actions CI/CD**.
- Tagging and pushing standardized container images to **Docker Hub** (`docker-frontend-2400032903` and `docker-backend-2400032903`).
- Orchestrating the full multi-tier topology (Frontend, Backend, and PostgreSQL database) using **Docker Compose**.

---

1. Why GreenRoots? The Domain and Problem Statement

Deforestation and climate degradation demand accountable, data-driven green initiatives. **GreenRoots** was conceived as an Afforestation Event Management System designed to eliminate spreadsheet chaos for environmental NGOs and campus green chapters.

The system manages tree plantation initiatives, capturing critical operational attributes:

- **Initiative Name:** Descriptive title of the drive (e.g., *Coastal Mangrove Restoration*, *Urban Canopy Project*).
- **Location / Botanical Site:** Geo-specific planting ground or sanctuary.
- **Trees Planted:** Verified numerical count of saplings planted.
- **Drive Status:** Lifecycle state indicator (*Ongoing* vs. *Completed*).

- **System-Generated ID:** Sequential, database-enforced primary key generated automatically by PostgreSQL (ensuring users never manually enter or manipulate identity keys).

Beyond basic CRUD, GreenRoots serves as a real-world SOA service model: a distinct, domain-specific service communicating with client applications exclusively through standardized HTTP/JSON REST endpoints.

---

2. Frontend Engineering: Clean UX and Responsive Interface

The user interface was built using **React 19** paired with **Bootstrap 5** and customized CSS variables, designed with an earthy forest aesthetic (deep forest greens, sage tints, and glassmorphic modal surfaces).

Key Frontend Capabilities:

- **Responsive Navigation Bar:** Accessible navigation with brand identity, live metrics, and direct links to API documentation.
- **Dynamic Event Directory:** Responsive grid displaying active plantation drives, status badges, tree volumes, and deletion triggers.
- **Client-Side Form Validation:** Enforces strict boundary checks (minimum character lengths, positive integer validation for sapling counts, and auto-trimmed inputs) with contextual error alerts.
- **Safe Identity Management:** The creation modal completely eliminates manual ID inputs, delegating identifier assignment entirely to the persistence layer.
- **Non-blocking Delete Confirmations:** Clean confirmation dialogs with asynchronous progress indicators preventing duplicate submissions.

📸 **[Insert Screenshot 1 — GreenRoots Main Page / Dashboard & Navbar]**

The GreenRoots landing interface, showcasing responsive navigation, ecological impact metrics, and plantation cards.

📸 **[Insert Screenshot 2 — Add/Update Plantation Event Modal & Directory View]**

Data capture form demonstrating client-side validation, completion toggles, and safe record submission.

---

3. Backend Architecture: Spring Boot REST Microservice

The backend was developed using **Spring Boot 3 / Java 21** following strict layered architecture principles under the designated base package:

```
com.klef.soa.docker
├── configuration/    ⟶ OpenAPI / Swagger UI documentation configs
├── controller/       ⟶ REST Controller exposing endpoints (/api/events)
├── entity/           ⟶ JPA Entity (TreePlantationEvent) mapped to PostgreSQL
├── repository/       ⟶ Spring Data JPA Repository
├── service/          ⟶ Business service interfaces
├── service.impl/     ⟶ Service business logic implementation
└── exception/        ⟶ GlobalExceptionHandler with custom RFC-compliant errors
```

Layered Separation of Concerns:

1. **Controller Layer (`TreePlantationEventController`):** Exposes clean RESTful endpoints (`GET`, `POST`, `PUT`, `DELETE`) mapping HTTP semantics to domain actions.
2. **Service Layer (`TreePlantationEventService`):** Implements business validation rules, ensuring data integrity before persistence.
3. **Repository Layer (`TreePlantationEventRepository`):** Leverages Spring Data JPA abstraction for optimized relational queries against PostgreSQL.
4. **Global Exception Handler:** Intercepts `ResourceNotFoundException` and validation faults, translating internal stack traces into user-friendly JSON error payloads.

Contract-First Verification with Swagger UI

To test backend operations independently of the frontend, **OpenAPI / Swagger UI** was integrated (`/swagger-ui/index.html`), allowing real-time inspection of API schemas, request models, and response status codes (`200 OK`, `201 Created`, `404 Not Found`).

> 📸 **[Insert Screenshot 3 — Swagger UI Showing REST Endpoints & Testing]**
> *Interactive Swagger documentation verifying CRUD operations against the Spring Boot service.*

---

4. The Core Moto: Dockerization & Multi-Stage Builds

The centerpiece of this project was **containerization**—eliminating the "it works on my machine" dilemma by bundling application code, runtime dependencies, and environment configurations into self-contained Docker images.

Both frontend and backend folders contain dedicated, isolated `Dockerfile` configurations utilizing **multi-stage builds** to maximize performance and minimize attack surfaces:

Backend Dockerfile (`backend/Dockerfile`)

The backend leverages a two-stage build. Rather than shipping a bulky 800MB Maven development image, the first stage builds the `.jar` using the Maven Wrapper (`mvnw`), and the second stage packages only the compiled bytecode inside a clean **Eclipse Temurin 21 JDK** runtime:

```
# Stage 1: Build the app using Maven Wrapper
FROM eclipse-temurin:21-jdk AS builder
WORKDIR /app
COPY mvnw .
COPY .mvn/ .mvn
COPY pom.xml ./
COPY src ./src
RUN chmod +x mvnw
RUN ./mvnw clean package -DskipTests

# Stage 2: Minimal runtime image
FROM eclipse-temurin:21-jdk
WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar
EXPOSE 8081
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Frontend Dockerfile (`frontend/Dockerfile`)

Similarly, the React application is built in a **Node 20 Alpine** environment, generating production-optimized, minified static bundles (`dist/`), which are then served using high-performance **Nginx Alpine**:

```
# Stage 1: Build React static bundle
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# Stage 2: Serve production assets via Nginx
FROM nginx:alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Additionally, `.dockerignore` files were configured in both directories to exclude `node_modules/` , `target/` , and `.git/` , reducing build context transfer sizes from hundreds of megabytes to mere kilobytes.

---

5. Automating the Pipeline with GitHub Actions CI/CD

To establish modern DevOps automation, I created a GitHub Actions workflow (`.github/workflows/docker-image.yml`) triggered automatically on every `push` to the `main` branch.

Pipeline Workflow Steps:

1. **Source Checkout:** Clones the latest commit from the repository using `actions/checkout@v4`.
2. **Buildx Environment:** Configures the advanced Docker Buildx engine via `docker/setup-buildx-action@v3`.
3. **Registry Authentication:** Authenticates securely against Docker Hub using encrypted repository secrets (`DOCKERHUB_USERNAME` and `DOCKERHUB_TOKEN`).
4. **Standardized Image Tagging & Publishing:** Builds and pushes images strictly tagged with my student roll number:
   - **Backend:** `dheeraj200579/docker-backend-2400032903:latest`
   - **Frontend:** `dheeraj200579/docker-frontend-2400032903:latest`

📸 **[Insert Screenshot 4 — Successful GitHub Actions CI/CD Run]**

GitHub Actions pipeline completing the automated build and push steps for both microservices.

---

6. Docker Hub Registry Publication

Once the pipeline concludes, the compiled artifacts reside in **Docker Hub**, functioning as an immutable container registry accessible anywhere globally.

This establishes true deployment portability: any host machine or cloud instance with Docker installed can pull and run GreenRoots without needing local installations of Java, Maven, Node.js, or npm.

📸 **[Insert Screenshot 5 — Docker Hub Repositories]**

Published public Docker images: docker-backend-2400032903 and docker-frontend-2400032903.

---

7. Multi-Container Orchestration with Docker Compose

Running a distributed application requires orchestrating multiple interconnected services. GreenRoots utilizes `docker-compose.yml` to define the multi-tier topology:

```yaml
services:
  # 1. Relational Database Layer
  postgres-db:
    image: postgres:16-alpine
    container_name: greenroots-postgres
    restart: always
    environment:
      POSTGRES_DB: greenroots_db
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: docker
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks:
      - greenroots-net
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d greenroots_db"]
      interval: 5s
      timeout: 5s
      retries: 5

  # 2. Spring Boot Backend Microservice
  backend:
    image: dheeraj200579/docker-backend-2400032903:latest
    build:
      context: ./backend
      dockerfile: Dockerfile
    container_name: greenroots-backend
    restart: on-failure
    ports:
      - "8081:8081"
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres-db:5432/greenroots_db
      SPRING_DATASOURCE_USERNAME: postgres
      SPRING_DATASOURCE_PASSWORD: docker
      SPRING_JPA_HIBERNATE_DDL_AUTO: update
    depends_on:
      postgres-db:
        condition: service_healthy
    networks:
      - greenroots-net

  # 3. React Frontend Service
  frontend:
    image: dheeraj200579/docker-frontend-2400032903:latest
    build:
      context: ./frontend
      dockerfile: Dockerfile
    container_name: greenroots-frontend
    restart: on-failure
    ports:
      - "3000:80"
    depends_on:
      - backend
    networks:
      - greenroots-net

networks:
  greenroots-net:
```

```
    driver: bridge

volumes:
  postgres_data:
```

Key Orchestration Features:

- **Service Dependency & Healthchecks:** The backend container waits for PostgreSQL to pass healthchecks (`pg_isready`) before launching, eliminating database connection refusal crashes.
- **Isolated Bridge Network (`greenroots-net`):** All containers communicate internally using Docker DNS service discovery (e.g., `postgres-db:5432`), isolating network traffic.
- **Volume Persistence (`postgres_data`):** Preserves tree plantation records across container restarts and updates.

Executing a single command boots up the entire application stack:

```
docker compose up -d
```

📸 **[Insert Screenshot 6 — Docker Compose / Running Containers via Terminal & Desktop]**

Terminal execution showing docker compose up -d and docker ps with all 3 containers healthy and active.

---

8. Complete Execution Demonstration

Source Code → GitHub Actions CI → Docker Hub → Docker Compose → Running Application

In the video demonstration, I showcase:

1. End-to-end full-stack interaction (creating, retrieving, and deleting plantation drives from the React UI).
2. Swagger UI endpoint testing.
3. Automated GitHub Actions CI workflow execution.
4. Container orchestration and logs verified via Docker CLI (`docker ps`, `docker compose logs`).

🎥 **[Insert the GreenRoots Execution Video Here]**

Video demonstrating UI operations, API validations, Docker Hub tags, and Docker Compose execution.

---

9. Key Takeaways & Reflection

- **SOA in Practice:** Realizing how decoupling the frontend from the backend enables independent scaling, polyglot development, and cleaner maintenance.
- **The Power of Containerization:** Understanding how Docker standardizes runtime environments, ensuring reproducibility across developer machines and cloud runners.
- **DevOps Efficiency:** Automating image creation and publishing with GitHub Actions demonstrates how continuous integration protects teams from manual packaging mistakes.
- **Architectural Discipline:** From enforcing database-generated IDs to configuring multi-stage Docker builds, thoughtful architectural decisions save countless hours of debugging downstream.

Acknowledgement

I would like to express my sincere gratitude to my course instructor, **[Faculty Name]**, for introducing us to Service-Oriented Architecture, containerization, and CI/CD concepts. The hands-on labs and guidance provided a comprehensive roadmap for mastering modern software deployment practices.

Source Code & Resources

- **GitHub Repository:** https://github.com/Dheeraj32903/GreenRoots-Docker-CICD
- **Docker Hub Images:**
  - `dheeraj200579/docker-backend-2400032903:latest`
  - `dheeraj200579/docker-frontend-2400032903:latest`